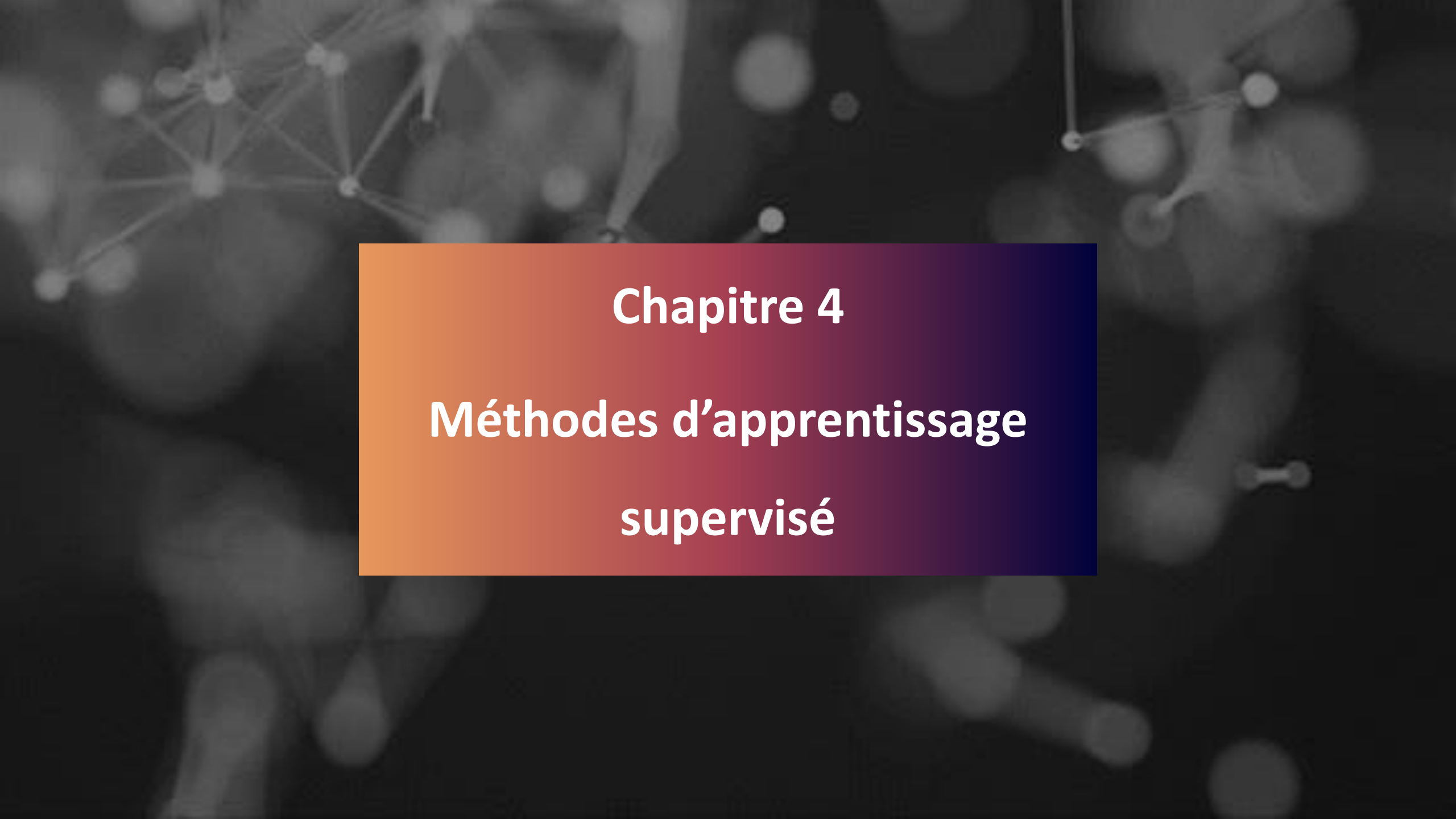




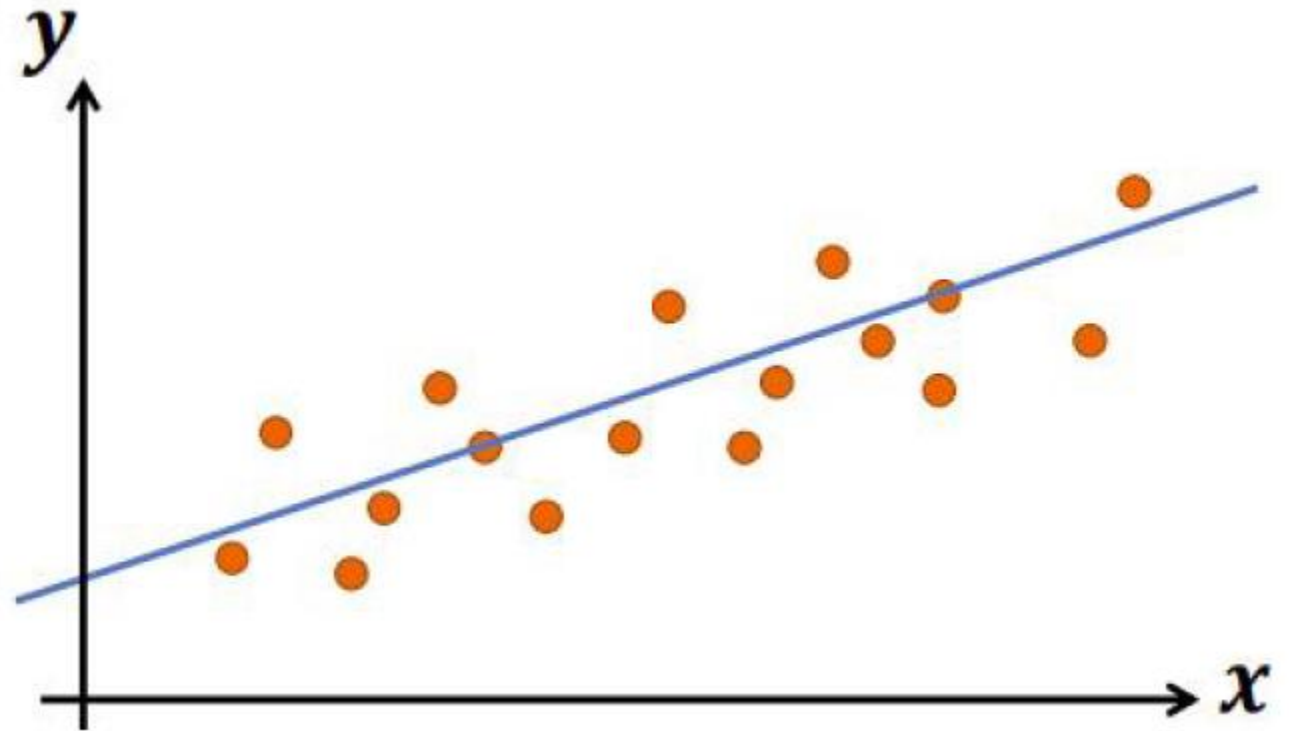
# **Chapitre 4**

## **Méthodes d'apprentissage supervisé**

# Régression linéaire

La régression linéaire a pour objectif de prédire la valeur d'une variable dépendante ( $y$ ) à partir d'une variable indépendante donnée ( $x$ ). Ainsi, cette technique de régression permet de déterminer une **relation linéaire** entre  $x$  (entrée) et  $y$  (sortie).

$$y = a x + b$$



# Régression linéaire

- ❖ Lorsque le nombre de l'entité indépendante est égal à 1, on parle de **régression linéaire simple** (ou univariée) et, dans le cas de plusieurs entités indépendantes, de **régression linéaire multiple** (multivariée). Dans le cas de la régression linéaire simple, l'objectif de l'algorithme est de trouver la meilleure équation linéaire  $y = aX + b$  où  $b$  est l'ordonnée à l'origine et  $a$ , le coefficient directeur (ou pente) de la droite. Par exemple, prédire une note à un examen (variable indépendante) en fonction du nombre d'heures de révisions (variable dépendante). En d'autres termes, les données d'une série d'observations sont représentées sous forme d'un nuage de points et l'on cherche à trouver une droite passant au plus près de ces points.
- ❖ Dans le cas de la régression linéaire multiple, plusieurs variables indépendantes sont utilisées  $y = b + a_1X_1 + \dots + a_nX_n$ . Par exemple, nous allons chercher à prédire le temps que va mettre un cycliste pour remporter une compétition, en fonction de son âge, du temps qu'il a réalisé à la précédente étape ...

# Régression linéaire

❖ Les prédictions en utilisant la Régression linéaire

Étant donné que la représentation est une équation linéaire, il suffit de résoudre l'équation pour un ensemble spécifique d'entrées pour faire des prédictions. Par exemple pour prédire le poids ( $y$ ) à partir de la taille ( $x$ ). La représentation du modèle de régression linéaire pour ce problème serait la suivante :

$$y = B_0 + B_1 * x_1 \text{ ou } \text{poids} = B_0 + B_1 * \text{taille}$$

Où  $B_0$  est le coefficient de biais et  $B_1$  est le coefficient pour la colonne de la taille. Après la phase de l'apprentissage du modèle nous pouvons introduire différentes valeurs de taille pour prédire le poids.

Par exemple, supposons  $B_0 = 0,1$  et  $B_1 = 0,5$ . Le poids d'une personne mesurant 182 centimètres serait selon le modèle 91,1 kg

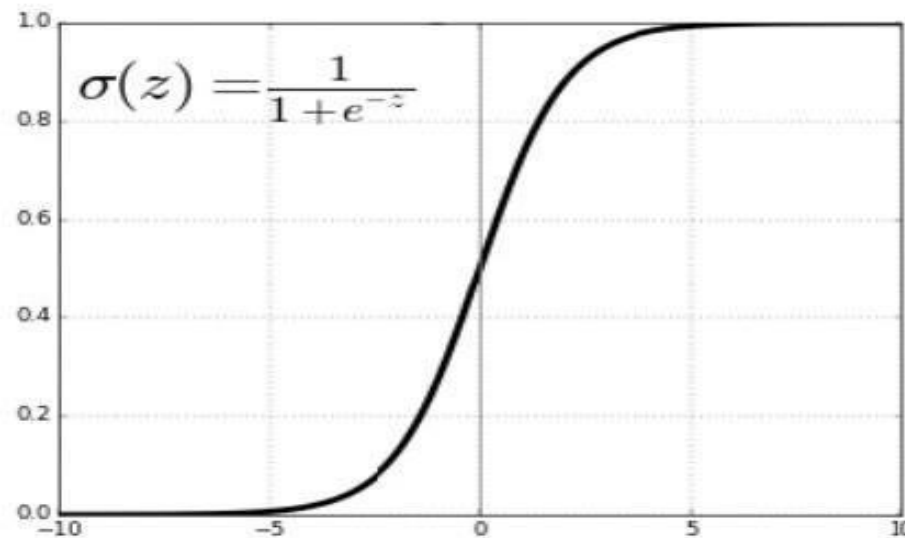
$$\text{poids} = 0,1 + 0,5 * 182$$

$$\text{poids} = 91,1$$

# Régression logistique

La régression logistique est un algorithme supervisé utilisé pour les problèmes de **classification** binaire. La régression logistique utilise une fonction logistique appelée sigmoïde ou courbe en S, définie sur  $\mathbb{R}$  à valeurs dans  $[0,1]$ . Elle s'écrit de la manière suivante :

$$\sigma(z) = \frac{1}{1+e^{-z}} \quad \text{ou} \quad \begin{cases} z \rightarrow \infty, \sigma(z) \rightarrow 1 \\ z \rightarrow -\infty, \sigma(z) \rightarrow 0 \\ z = 0, \sigma(z) = 0.5 \end{cases}$$



Fonction sigmoïde

# Régression logistique

❖ La classification en utilisant la Régression logistique

Pour prédire si un échantillon  $x$  appartient à une classe donnée (classe positive)  $y=1$ , on calcule sa probabilité en utilisant l'équation précédente. Ensuite, on utilise un seuil sur cette probabilité pour décider.

Par exemple, on peut utiliser le seuil 0.65. Dans ce cas:

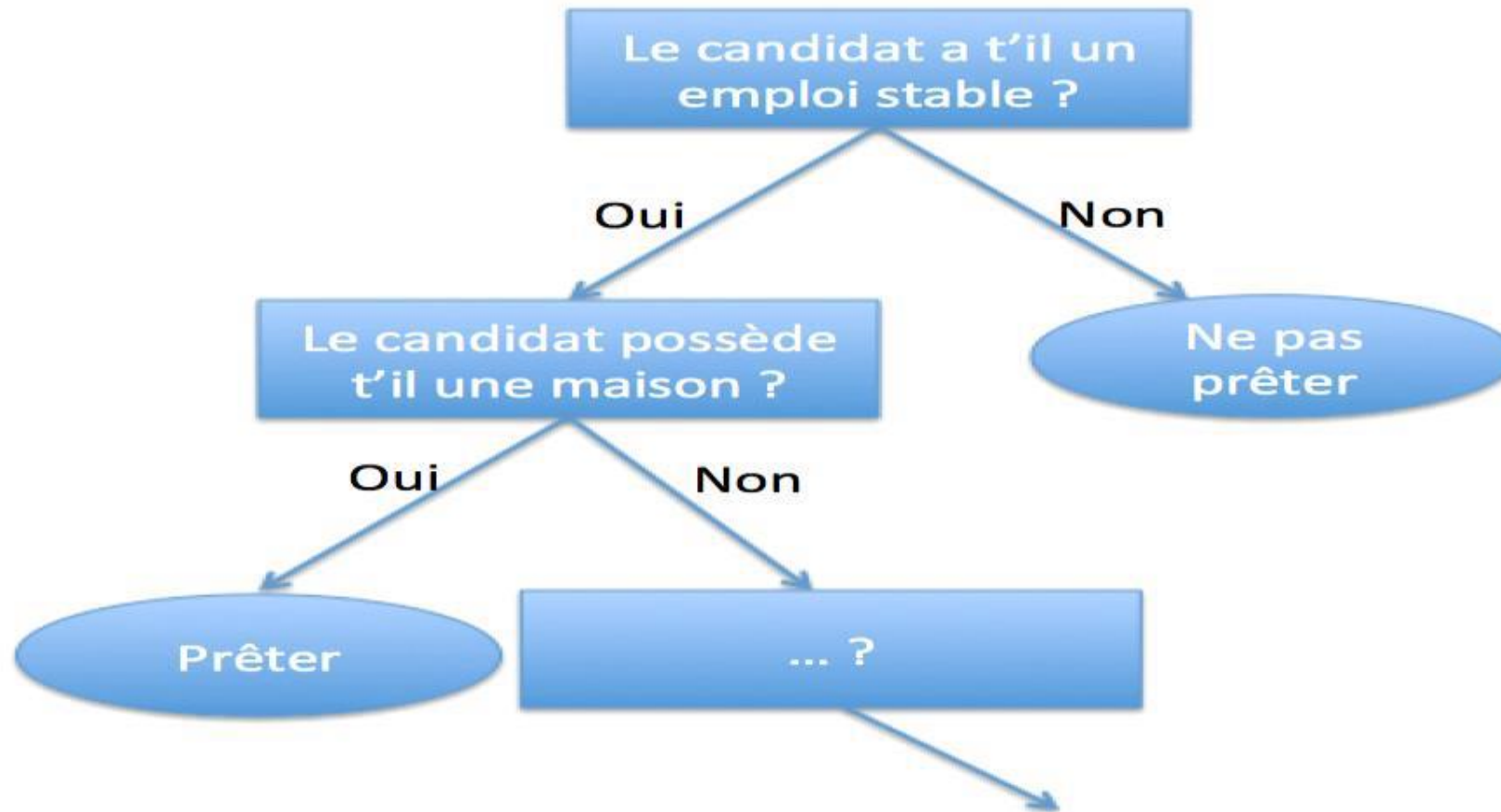
- Si  $p(y=1 | x) \geq 0.65$  donc classe positive
- Sinon classe négative

## Arbres de décision

Un arbre de décisions est un algorithme d'apprentissage supervisé non paramétrique, qui est **utilisé à la fois pour les tâches de classification et de régression**. Il a une structure hiérarchique, une structure arborescente, qui se compose d'un nœud racine, de branches, de nœuds internes et de nœuds feuille qui n'ont pas de descendant. Un arbre de décision commence par un nœud racine d'où découlent plusieurs résultats possibles. Chacun de ces résultats mène à d'autres nœuds, d'où émanent d'autres possibilités. Le schéma ainsi obtenu rappelle la forme d'un arbre.

# Arbres de décision

- Exemple : Prêt bancaire



# Arbres de décision

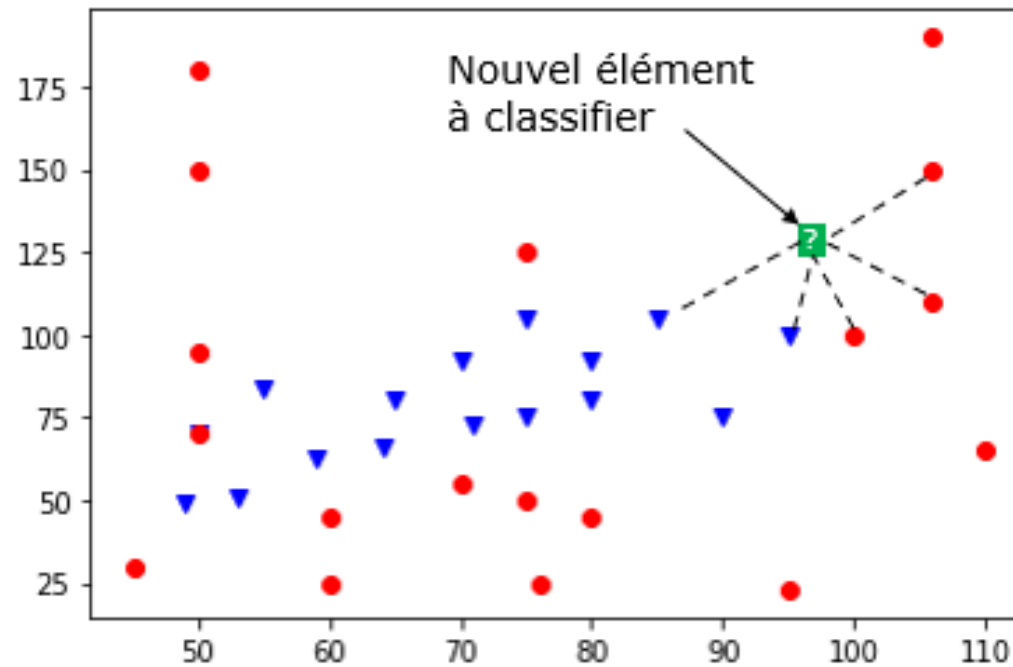
## ❖ Principe de la construction :

Au départ, les points de la base d'apprentissage sont tous placés dans le nœud racine. Chaque nœud est coupé (opération split) donnant naissance à plusieurs nœuds descendants.

1. **Sélection de la meilleure caractéristique** : À chaque nœud de l'arbre, on choisit la caractéristique la plus informative pour diviser les données. Cette décision repose sur des critères comme l'indice de Gini, l'entropie, ou la variance.
2. **Division des données** : Les données sont divisées en sous-ensembles selon les valeurs de la caractéristique sélectionnée. Le but est de rendre les sous-ensembles aussi homogènes que possible en termes de la variable cible.
3. **Répétition du processus** : On répète cette division récursive jusqu'à ce qu'un critère d'arrêt soit atteint (profondeur maximale de l'arbre, nombre minimal d'éléments dans un nœud, ou pureté suffisante des sous-ensembles).
4. **Prédiction des feuilles** : Une fois la construction de l'arbre terminée, chaque nœud feuille contient la prédiction finale, soit la classe majoritaire (en classification) ou la moyenne des valeurs (en régression).

## K-Plus Proches Voisins (KNN)

L'algorithme des k plus proches voisins, en anglais k-NN pour « k Nearest Neighbors » est un algorithme d'apprentissage supervisé qui permet à la fois de résoudre un problème de **classification** et de **régression**. L'algorithme de k plus proches voisins ne nécessite pas de phase d'apprentissage à proprement parler.



# K-Plus Proches Voisins (KNN)

## ❖ Pseudo code de K-NN

Pour une nouvelle observation inconnue en entrée dont on veut prédire sa variable de sortie:

**Etape 1** : Calculer toutes les distances entre cette observation en entrée et les autres observations du jeu de données,

**Etape 2** : Conserver les k observations du jeu de données qui sont les plus « proches » de l'observation à prédire. Pour déterminer si une valeur est plus ou moins « proche » d'une autre, on utilise une fonction de similarité reposant sur un calcul de distance.

**Etape 3** : Prendre les valeurs des observations retenues :

-Si on effectue une régression, l'algorithme calcule la moyenne (ou la médiane) des valeurs des observations retenues,

-Si on effectue une classification, l'algorithme assigne le label de la classe majoritaire à la donnée qui était inconnue.

**Etape 4** : Retourner la valeur calculée dans l'étape 3 comme étant la valeur qui a été prédite par l'algorithme pour l'observation en entrée qui était inconnue.

**Remarque** : Lorsque nous travaillons avec l'algorithme, il est utile de réduire le bruit qui peut être dans l'ensemble de données afin d'obtenir des résultats plus précis.

# K-Plus Proches Voisins (KNN)

## ❖ Métriques d'évaluation de la similarité entre les observations

Pour l'algorithme K-NN, le choix du nombre  $k$  et le choix de la fonction de similarité sont des étapes qui peuvent conduire à une forte variabilité des résultats. La fonction qui calcule la distance entre deux observations estime l'affinité entre les observations comme ceci : Plus deux points sont proches l'un de l'autre, plus ils sont similaires. Il existe de nombreuses fonctions de similarité dans la littérature. Parmi les fonctions de similarité les plus connues, on cite la distance euclidienne et la distance de Manhattan.

### 1- Distance euclidienne :

Dans un espace à  $n$  dimensions, la distance euclidienne entre les points  $X(x_1; x_2; \dots; x_n)$  et  $Y(y_1; y_2; \dots; y_n)$  est :

$$d(X, Y) = \sqrt{\sum_{i=1}^n (y_i - x_i)^2}$$

### 2- Distance de Manhattan :

Appelée aussi taxi-distance, est la distance entre deux points parcourue par un taxi lorsqu'il se déplace dans une ville où les rues sont agencées selon un réseau ou quadrillage, à l'image de Manhattan. La distance de Manhattan entre A et B est définie par :

$$d(A, B) = |X_B - X_A| + |Y_B - Y_A|$$

# Algorithmes d'ensemble

Un algorithme d'ensemble fait référence à un groupe (ou ensemble) d'apprenants de base, ou de modèles, qui travaillent collectivement à l'obtention d'une meilleure prédiction finale. Un modèle individuel, également appelé modèle de base ou apprenant faible (weak learner), peut présenter des performances médiocres en raison d'une variance ou d'un biais élevé. Par faible, nous entendons un apprenant dont le taux d'erreur est légèrement meilleur que celui d'une règle purement aléatoire. Cependant, lorsque des apprenants faibles sont agrégés, ils peuvent former un apprenant fort, car leur combinaison réduit le biais ou la variance, ce qui améliore les performances du modèle.

Il existe trois grandes familles principales :

**Bagging** : Entraîner plusieurs modèles **indépendants** sur des sous-échantillons aléatoires des données. Exemple d'algorithme : forêts aléatoires.

**Boosting** : Construire les modèles de manière **séquentielle**, chaque nouveau modèle corrigeant les erreurs des précédents.

**Stacking** : Combiner plusieurs modèles (souvent de nature différente) via un métamodèle qui apprend à prédire à partir de leurs sorties.

Dans le bagging, les apprenants faibles sont entraînés en parallèle, tandis que dans le boosting, l'apprentissage se fait de manière séquentielle. Autrement dit, une série de modèles est construite, et à chaque nouvelle itération, les poids des données mal classifiées dans le modèle précédent sont augmentés. Cette redistribution des poids permet à l'algorithme de se concentrer sur les points de données qui nécessitent une attention particulière afin d'améliorer ses performances.

# Algorithmes d'ensemble

## Bagging

Le Bagging est une technique en intelligence artificielle qui consiste à assembler un grand nombre d'algorithmes avec de faibles performances individuelles pour en créer un beaucoup plus efficace. Les algorithmes de faible performance sont appelés les **weak learners** et le résultat obtenu **strong learner**.

Les weak learners peuvent être de différentes natures et avoir des performances variées, mais ils doivent être **indépendants** les uns des autres.

L'assemblage des weak learners en strong learner se fait par votation. Moyenne (pour la régression) ou vote majoritaire (pour la classification).

# Forêts aléatoires

- ❖ Les forêts aléatoires sont une extension du bagging, dans laquelle la méthode est enrichie par la **sélection aléatoire de caractéristiques à chaque nœud**. Cela permet de réduire la corrélation entre les arbres et d'améliorer la robustesse du modèle global.
- ❖ Une forêt aléatoire est composée de plusieurs arbres de décision, entraînés de manière indépendante **sur des sous-ensembles** de data set d'apprentissage (méthode de bagging). Chaque arbre produit une estimation, et c'est la combinaison des résultats qui va donner la prédiction finale. Elle peut être utilisée pour des problèmes de **classification ou de régression**. L'algorithme de forêt aléatoire repose sur le **bagging** et le **feature randomness**, également appelé feature bagging qui assure une faible corrélation entre les arbres de décision. A la fin, tous les arbres de décisions indépendants sont assemblés. La prédiction faite par le random forest pour des données inconnues est alors la moyenne (ou le vote, dans le cas d'un problème de classification) de tous les arbres.

# Forêts aléatoires

❖ Étapes pour construire une forêt aléatoire :

## 1. Initialisation

- **Entrée :**
  - Ensemble d'entraînement  $D$  (avec  $n$  exemples et  $m$  caractéristiques).
  - Nombre d'arbres  $T$  à entraîner.
  - Nombre de caractéristiques  $k$  à considérer pour chaque division.

## 2. Construction de chaque arbre

Tirer un sous-échantillon aléatoires  $D_{rand}$  depuis l'ensemble d'entraînement  $D$ .

Construire l'arbre de décision  $t$  sur  $D_{rand}$  :

- Pour chaque nœud de l'arbre :
  - Sélectionner aléatoirement  $k$  caractéristiques parmi les  $m$  disponibles.
  - Choisir la meilleure caractéristique parmi ces  $k$  selon un critère de division (par exemple, l'indice de Gini pour la classification).
  - Diviser les données en sous-ensembles selon la meilleure caractéristique sélectionnée.
- Répéter jusqu'à atteindre un critère d'arrêt, tel que :
  - Une profondeur maximale de l'arbre.
  - Un nombre minimal d'exemples dans chaque nœud.
  - Un seuil de pureté suffisant pour les nœuds.

Ajouter l'arbre  $t$  à la forêt.

# Algorithmes de Boosting

## Boosting

Les algorithmes de Boosting se basent sur le même principe que ceux de Bagging. La différence apparaît lors de la création des weak learner.

Pour le Boosting, les algorithmes ne sont plus indépendants. Au contraire, chaque weak learner est entraîné pour corriger les erreurs des weak learner précédents. En d'autres termes, chaque modèle tente de compenser les faiblesses de son prédécesseur.

Le principe général du boosting consiste à construire une famille de modèles qui sont ensuite agrégés par une **moyenne pondérée** (en régression) ou un vote à la majorité (en classification).

Parmi les algorithmes de Boosting les plus célèbres : Adaboost, Gradient Boosting et XGBoost.

Les modèles sont construits de manière récursive : chaque modèle est une version adaptative du précédent en donnant plus de poids aux observations mal prédites. Le modèle construit à l'étape  $k$  concentrera donc ses efforts sur les observations mal ajustées par le modèle à l'étape  $k - 1$ .

# Algorithmes de Boosting

## AdaBoost (Adaptative Boosting Algorithm)

L'algorithme adaboost a été développé par Freund & Schapire (1997). Il s'agit de l'un des algorithmes le plus populaire appartenant à la famille des algorithmes de boosting.

La technique utilisée par AdaBoost consiste à faire en sorte que le nouveau prédicteur corrige son prédécesseur. Un classificateur AdaBoost est un méta-estimateur qui commence par ajuster un classificateur sur l'ensemble de données d'origine, puis ajuste des copies supplémentaires du classificateur sur le même ensemble de données, mais en ajustant les poids des instances mal classées de sorte que les classificateurs suivants se concentrent davantage sur les cas difficiles.

Les weak learners d'AdaBoost sont généralement des arbres décisionnels à seulement 2 branches et 2 feuilles (appelés **stumps**), mais on peut utiliser d'autres types de classificateur à condition de : pouvoir gérer des poids d'échantillons, et d'avoir un classifieur non trop puissant, sinon le boostage perd de son intérêt.

# Algorithme AdaBoost

**Entrée** Les éléments nécessaires sont

- un échantillon  $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)$
- un ensemble de règles faibles
- le nombre  $M$  d'itérations

**Initialisation** on se donne des poids  $(w_1, \dots, w_n)$  uniformes

$$w_i \leftarrow \frac{1}{n}, i = 1, \dots, n,$$

qui vérifient évidemment  $w_1 + \dots + w_n = 1$ .

**Itération** pour  $m = 1$  à  $M$

1. ajuster un classifieur faible  $g_m(\mathbf{x})$  sur l'échantillon pondéré par les poids  $w_i$
2. calculer le taux d'erreur

$$\epsilon_m \leftarrow \sum_{i=1}^n w_i \mathbf{1}_{\{y_i \neq g_m(\mathbf{x}_i)\}}$$

3. calculer le poids de l'itération  $m$  :

$$\alpha_m \leftarrow \log \sqrt{(1 - \epsilon_m) / \epsilon_m}$$

4. mettre à jour les poids des observations

$$\begin{aligned} w_i &\leftarrow \frac{1}{Z_m} w_i \exp[-\alpha_m y_i g_m(\mathbf{x}_i)] \\ &= \frac{1}{Z_m} w_i \times \begin{cases} e^{-\alpha_m}, & \text{si } y_i = g_m(\mathbf{x}_i), \\ e^{\alpha_m} & \text{sinon.} \end{cases} \end{aligned}$$

**Sortie** c'est le signe de la combinaison linéaire

$$\hat{g}_M(\mathbf{x}) = \text{sign} \sum_{m=1}^M \alpha_m g_m(\mathbf{x})$$

# Algorithme AdaBoost

Exemple : on veut prédire si un passager du Titanic va survivre ou non selon ses caractéristiques.

| Sexe | Age | Classe | Survie |
|------|-----|--------|--------|
| 1    | 30  | 3      | 0      |
| 2    | 24  | 3      | 1      |
| 1    | 10  | 1      | 1      |
| 2    | 16  | 2      | 1      |
| 1    | 67  | 1      | 0      |

# Algorithme AdaBoost

Etape 1 : initialisation des poids.

$$w_i \leftarrow \frac{1}{n}, i = 1, \dots, n,$$

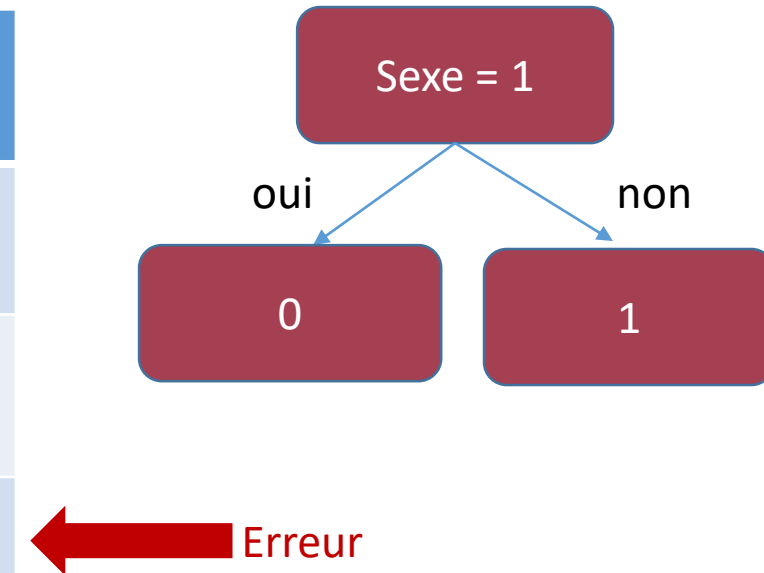
| Sexe | Age | Classe | Survie | Poids |
|------|-----|--------|--------|-------|
| 1    | 30  | 3      | 0      | 1/5   |
| 2    | 24  | 3      | 1      | 1/5   |
| 1    | 10  | 1      | 1      | 1/5   |
| 2    | 16  | 2      | 1      | 1/5   |
| 1    | 67  | 1      | 0      | 1/5   |

# Algorithme AdaBoost

Etape 2 : Construction du premier Stump.

| Sexe | Age | Classe | Survie |
|------|-----|--------|--------|
| 1    | 30  | 3      | 0      |
| 2    | 24  | 3      | 1      |
| 1    | 10  | 1      | 1      |
| 2    | 16  | 2      | 1      |
| 1    | 67  | 1      | 0      |

| Poids |
|-------|
| 1/5   |
| 1/5   |
| 1/5   |
| 1/5   |
| 1/5   |



# Algorithme AdaBoost

Etape 3 : calculer le taux d'erreur.  $\epsilon_m \leftarrow \sum_{i=1}^n w_i \mathbf{1}_{\{y_i \neq g_m(\mathbf{x}_i)\}} = 1/5$  pour cet exemple

Etape 4 : calculer le poids de l'arbre  $= \frac{1}{2} \log\left(\frac{1 - \text{taux erreur}}{\text{taux erreur}}\right) = \frac{1}{2} \log\left(\frac{1 - (\frac{1}{5})}{(\frac{1}{5})}\right) = 0,69$

Donc le poids du premier stump est 0,69

# Algorithme AdaBoost

Etape 5 : Calculer les nouveaux poids.

| Sexe | Age | Classe | Survie |
|------|-----|--------|--------|
| 1    | 30  | 3      | 0      |
| 2    | 24  | 3      | 1      |
| 1    | 10  | 1      | 1      |
| 2    | 16  | 2      | 1      |
| 1    | 67  | 1      | 0      |

| Poids | Normalisation  |                       |
|-------|----------------|-----------------------|
|       | Nouveaux Poids | Nouveaux Poids (norm) |
| 1/5   | 0,1            | 0,125                 |
| 1/5   | 0,1            | 0,125                 |
| 1/5   | 0,4            | 0,5                   |
| 1/5   | 0,1            | 0,125                 |
| 1/5   | 0,1            | 0,125                 |

Refaire ces étapes  $M$  fois

Fin